

Ip Http DNS

Pedro Gutiérrez La Ossa

2 ASIR

índice

TCP/IP

HTTP

DNS

Pruebas y comandos de red

Conclusión

1. ¿Qué significa TCP/IP y para qué se utiliza?

TCP/IP significa Transmission Control Protocol / Internet Protocol. Es el conjunto de protocolos que permite que los dispositivos se comuniquen en una red

2. ¿Cuáles son las capas del modelo TCP/IP? ¿En qué se diferencian del modelo OSI?

El modelo TCP/IP tiene 4 capas:

Acceso a red ,Internet,Transporte,Aplicación

Y el modelo OSI tiene 7 capas

3. ¿Por qué es importante la capa de transporte (TCP) en las comunicaciones de red?

Porque TCP garantiza que los datos lleguen correctamente .

4. Explica qué hace el comando `netstat -anp | grep 3000`. Pon un ejemplo.

Muestra las conexiones y puertos de red activos y con el comando `grep` busca solo los del puerto 3000

Por ejemplo si usamos un servidor en el puerto 3000 podemos ver si está escuchando

5. ¿Qué es el protocolo HTTP?

HTTP (HyperText Transfer Protocol) es el protocolo utilizado para intercambiar información

6. ¿Cómo se estructura una solicitud HTTP?

Una solicitud HTTP contiene principalmente:

1. Línea de petición: método, URL y versión HTTP.
2. Cabeceras: información adicional sobre la solicitud.
3. Cuerpo: datos enviados al servidor, principalmente en métodos como POST.

7. ¿Qué diferencia hay entre los métodos GET y POST en HTTP?

GET: solicita información al servidor. Los datos suelen enviarse en la URL.

POST: envía datos al servidor, normalmente en el cuerpo de la solicitud. Se utiliza, por ejemplo, para enviar formularios.

8. ¿Cuál es la función principal del sistema DNS?

DNS (Domain Name System) se encarga de traducir nombres de dominio

9. ¿Qué hace el comando nslookup?

Permite consultar un servidor DNS para obtener información sobre un dominio, como su dirección IP.

10. ¿Cómo se relaciona el comando ping con DNS?

Cuando hacemos ping a un nombre de dominio, primero se realiza una resolución DNS para obtener su dirección IP. Después, ping envía paquetes a esa IP para comprobar si el dispositivo responde.

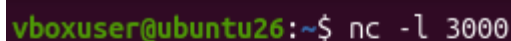
PRACTICA 1

usamos el comando que está en la imagen este sirve para establecer conexiones tcp o udp

NC es la herramienta que usamos (net cat)

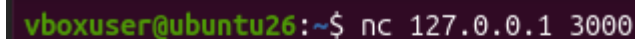
-L para listening

3000 el puerto que usamos



```
vboxuser@ubuntu26:~$ nc -l 3000
```

usamos nc 127.0.0.1 en el puerto 3000 para establecer la conexion



```
vboxuser@ubuntu26:~$ nc 127.0.0.1 3000
```

escribimos algo para ver que estan conectados

```
vboxuser@ubuntu26:~$ nc 127.0.0.1 3000
hola caracola pedrito

```

```
vboxuser@ubuntu26:~$ nc -l 3000
hola caracola pedrito

```

Iniciamos wireshark

```
root@ubuntu26:/home/vboxuser# wireshark
** (wireshark:8438) 15:04:44.374021 [Capture MESSAGE] -- Capture Start ...
** (wireshark:8438) 15:04:44.407783 [Capture MESSAGE] -- Capture started
** (wireshark:8438) 15:04:44.407882 [Capture MESSAGE] -- File: "/tmp/wireshark
LoopbackBZT2V3.pcapng"
** (wireshark:8438) 15:04:55.486552 [GUI WARNING] -- dbus reply error: [ "org.
reedesktop.DBus.Error.NoReply" ] "Did not receive a reply. Possible causes incl
de: the remote application did not send a reply, the message bus security polic
blocked the reply, the reply timeout expired, or the network connection was br
ken."
** (wireshark:8438) 15:04:55.486635 [GUI WARNING] -- dbus reply error: [ "org.
reedesktop.DBus.Error.NoReply" ] "Did not receive a reply. Possible causes incl
de: the remote application did not send a reply, the message bus security polic
blocked the reply, the reply timeout expired, or the network connection was br
ken."
** (wireshark:8438) 15:04:55.486772 [GUI WARNING] -- Failed to register with h
st portal QDBusError("org.freedesktop.DBus.Error.NoReply", "Did not receive a r
ply. Possible causes include: the remote application did not send a reply, the
essage bus security policy blocked the reply, the reply timeout expired, or the
network connection was broken.")

```

seleccionamos loopback y encontramos conexiones

```
any
Loopback: lo
bluetooth-monitor

```

filtramos por el puerto 3000 tcp

```
tcp.port == 3000

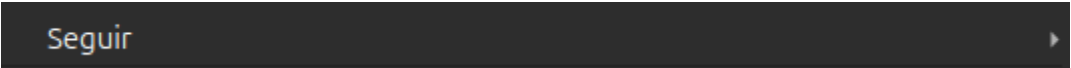
```

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	89	3000 → 41664 [PSH,
2	0.000044974	127.0.0.1	127.0.0.1	TCP	66	41664 → 3000 [ACK]

```
tcp.port == 3000

```

hacemos click derecho y seleccionamos seguir



Seguir

una vez dentro podemos ver el contenido del mensaje



hola caracola pedrito

PRACTICA 2

este comando `curl` realiza una petición HTTP GET al servidor de la página web y muestra la respuesta

```
vboxuser@ubuntu26:~$ curl http://www.google.com
```

```
Caution: You are using the Snap version of curl.  
Due to Snap's sandbox nature, this version has some limitations.  
For example, it may not be able to access hidden folders in your home directory  
or other restricted areas of the os.
```

```
Which means you may encounter errors when using snap curl to download and execute some script.
```

```
For those cases, you might want to use the native curl package.  
For details, see: https://github.com/boukendesho/curl-snap/issues/1
```

```
To stop seeing this message, run the following command:  
$ curl.snap-acked
```

```
<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="es">  
><head><meta content="Google.es permite acceder a la información mundial en castellano, catalán, gallego, euskara e inglés." name="description"><meta content="r  
oodp, " name="robots"><meta content="text/html; charset=UTF-8" http-equiv="Content-Type"><meta content="/images/branding/googleg/1x/googleg_standard_color_128dp  
.png" itemprop="image"><title>Google</title><script nonce="ews5m56qfJ-H K0aCpiq"
```

Lo que hace este comando es comprobar que el servidor responde

```
)();</script></body></html> vboxuser@ubuntu26:~$ GET -USE http://www.google.com
GET http://www.google.com
User-Agent: lwp-request/6.81 libwww-perl/6.81

200 OK
Cache-Control: private, max-age=0
Connection: close
Date: Thu, 24 Sep 2026 15:25:33 GMT
Accept-Ranges: none
Server: gws
Vary: Accept-Encoding
Content-Type: text/html; charset=ISO-8859-1
Expires: -1
Client-Date: Thu, 24 Sep 2026 15:25:33 GMT
Client-Peer: 142.251.152.119:80
Client-Response-Num: 1
Client-Transfer-Encoding: chunked
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src
'nonce-U5yRKpUNP5oW0UL6k5MoZw' 'strict-dynamic' 'report-sample' 'unsafe-eval'
'unsafe-inline' https: http;;report-uri https://csp.withgoogle.com/csp/gws/other
-hp
Set-Cookie: AEC=Aaa9EJri3YWB3oFZTENLOQ_01gnDVFS-qxUzo5THmRK1QqK4Wv6CpTY7Fko; exp
ires=Tue, 23-Mar-2027 15:25:33 GMT; path=/; domain=.google.com; Secure; HttpOnly
```

nslookup consulta el servidor DNS para averiguar qué dirección IP corresponde al dominio `www.google.com`.

```
vboxuser@ubuntu26:~$ nslookup www.google.com
Server:      127.0.0.53
Address:     127.0.0.53#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.154.119
Name:   www.google.com
Address: 142.251.153.119
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 142.251.152.119
Name:   www.google.com
Address: 142.251.151.119
Name:   www.google.com
Address: 142.251.157.119
Name:   www.google.com
Address: 2001:4860:4860:7700::
Name:   www.google.com
```

Ping comprueba la conectividad con el servicio

```
vboxuser@ubuntu26:~$ ping www.google.com  
PING www.google.com (142.251.150.119) 56(84) bytes of data.  
■
```

Conclusión

En esta práctica hemos aprendido cómo funcionan algunos protocolos de red y cómo comprobar conexiones mediante distintos comandos de Linux.